



System Programming in C

Course developed by: Ilan Gronau and Sara H. Geizhals

Lecture overview

- Brief intro to course ←
- Course server
- File system basics
- Navigating file system (`pwd`, `cd`, `ls`)
- Manipulating files and directories
(`chmod`, `cp`, `mv`, `rm`, `mkdir`, `rmdir`)

Course Overview

In this course we dive **deeper into programming principles**:

Program structure

- Data representation – type / scope / storage classes
- Memory organization of programs – stack vs. heap

Memory management

- Working with pointers
- Dynamic memory allocation
- Implementing objects using C modules (toward OOP...)

Course Overview

In this course we learn **new programming skills**:

Linux environment

- Working with UNIX-based OS (remote server and local VM)
- Using text as interface – Linux text parsing tools
- Scripting

Programming in C

- Type conversions & operators
- Working with arrays / pointers
- How to structure your program into files (modules)?

Course Overview

Where will I use these skills?

Linux and C programming provide an unbeatable combination for performance computing where we need:

- Speed
- Reliability (no crashes)

Very popular in:

- Systems-level programming
- High-performance scientific computing

For more on the joint history of C and Linux see:

- slides under “General Resources” on [Piazza Resources](#)
- video guide on the course’s Panoptro folder

Course structure / syllabus

Linux (lectures 1-4)

- Linux file system, text parsing, shell scripting

Programming in C (lectures 5-9)

- Data representation, functions and organization of stack
- Pointers, memory allocation and management

OOP in C (lectures 10-12)

- Structures, modules
- Projects with multiple source files

Course requirements

Grading policy

- Final Grade = 80% final exam + 20% homework exercises
- Additionally: to pass the course, you must pass ($\geq 60\%$) both components. This means:
 - To be eligible to take the final exam, you must have an average passing grade on the homework exercises.
 - To pass the course, you must pass the final exam ($\geq 60\%$) – either moed aleph or moed bet.

HW exercises – philosophy and policy

- 6 programming exercises; each covers a different topic
- Individually solving the exercises is the best way to learn the material and reach the final exam prepared
- All exercises count equally in final grade ($3\frac{1}{3}$ points each)
- Unsubmitted exercise = grade of 0
- Better to submit a partial solution than skip entirely

HW exercises – sharing ideas, not code

- You should work and submit HW exercises in partners (working alone is okay but not encouraged)
- Start by solving individually and then merge into one combined solution.
- You may discuss the exercise with other students, but make sure you are **not sharing code**
 - If you put your code in a shared space, you're sharing code
 - If you send or receive a source file or copied text from a source file, you're sharing code
 - If you are writing your code while having someone else's code in front of you, you're sharing code

HW exercises – sharing ideas, not code

- You should work and submit HW exercises in partners (working alone is okay but not encouraged)
- Start by solving individually and then merge into one combined solution.
- You may discuss the exercise with other students, but make sure you are **not sharing code**
- The course graders periodically look for **suspiciously similar bits of code** in your solutions. Finding these is easier than you think!

HW exercises – working on the exercise

- Exercises typically allocated 2 weeks (HW #1 typically allocated 1 week)
- We typically publish the next exercise a few days before the deadline of the previous exercise. Use this time wisely.

Examples:

- HW #1 published Thurs 19/3, due Sun 29/3
- HW #2 published Thurs 26/3, due Sun 12/4
- HW #3 published Thurs 16/4, due Sun 3/5
- Exercises contain detailed instructions + guidelines for validation/checking
- Make sure you save enough time for validation before submission

HW exercises – submission

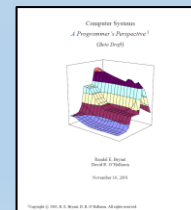
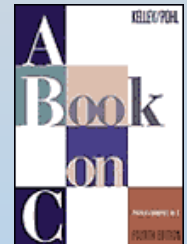
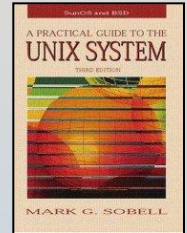
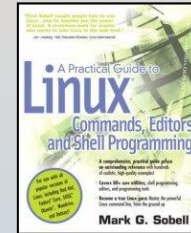
- Submission is done on the course server (`sysprog.runi.ac.il`)
- Detailed guide on checking your solution and submitting it in the **Homework submission guidelines** document on [Piazza Resources](#) (see general Resources)
- **check_ex** script for checking your solution before submission will give you instant feedback on errors
- **submit_ex** script for submitting your solution
- Late submission:
 - Extension requests will be considered **in special cases** and only if submitted **> 48 hours before deadline**
 - If your extension request is not granted, you will be penalized **10 points for every 24 hours**
 - If you are in **מילואים** and can't make the deadline, let us know ASAP

Other ways to practice material

- The course has a one-hour recitation (תרגולים) devoted to demonstrations and class exercises.
- We conduct **class exercises** in many lectures. A laptop is not required but will be helpful.

Course books

- Mark G. Sobell,
A Practical Guide to the UNIX System (3rd Edition)
- Mark G. Sobell,
*A Practical Guide to the Linux Commands,
Editors, and Shell Programming*
- Al Kelley and Ira Pohl,
A Book on C: Programming in C (4th Edition)
- Brian Kernighan and Dennis Ritchie,
The C Programming Language (2nd Edition)
- Randal E. Bryant, David R. O'Hallaron,
Computer systems: a programmer's perspective



Course website

- Most correspondence will be done through [Piazza](#)
- If you didn't receive an invitation by email, then email one of the instructors

piazza

CS 3144 Setup Q & A Resources Statistics Manage Class

Reichman University - Spring 2024

CS 3144: System programming in C

Syllabus

Course Information Staff Resources

Description

System programming in C (CS3144) provides an introduction to procedural programming and work in a UNIX/Linux environment. The course teaches basic topics in the C programming language, such as memory structure, memory management, pointers, and design of procedural programs. In parallel, we will cover fundamental topics in work in a Linux environment, such as managing permissions, working with text files, and programming in shell scripts. The course will involve programming assignments in C in a Linux environment.

General Information

Lectures

Mon 14:45 - 17:15 Raz (Hebrew), SL201
 --> Zoom link: <https://runi-ac-il.zoom.us/j/81983670695>

Mon 17:30 - 20:00 Sara (English), Hangar class H1
 --> Zoom link: <https://runi-ac-il.zoom.us/j/83908848870>

Tue 17:30 - 20:00 Raz (Hebrew), Hangar aud H2
 --> Zoom link: <https://runi-ac-il.zoom.us/j/88060176364>

Note: zoom is used in lectures for recording and for conducting class polls. Joining the lecture remotely will not be allowed as per university policy.

Recitations

Announcements

Welcome to CS3144 - System programming in C
 3/23/2024, 7:16:44 PM

Hello everyone,

You are receiving this email because you are registered to the course CS3144 System programming in C (תכנות מערכות בשפת C).

We will be communicating with you through this Piazza site and [View More](#) · [View on Piazza](#)

Message for Tuvia Students
 3/22/2024, 4:44:36 PM

Hi,

My name is Tuvia and i am your teacher assistant. We will have the lesson every Wednesday in semester B 2024. You can find me by mail : tovia.rosenbeg@runi.ac.il

Until our first meeting , please download putty and install in on your

Contact us

- Posts through the Piazza website
- Email:
 - Sara H. Geizhals shgeizhals@gmail.com
 - Daniel Karalnik daniel.karalnik@post.runi.ac.il
 - Raz Lin raz.lin@post.runi.ac.il
 - Liam Tal liam.tal@post.runi.ac.il
- Office hours – locations and times are listed on [Piazza](#)

Lecture overview

- Brief intro to course
- **Course server ←**
- File system basics
- Navigating file system (**pwd, cd, ls**)
- Manipulating files and directories
(**chmod, cp, mv, rm, mkdir, rmdir**)

Accessing Linux

- Work in this course will be done on RUNI's Linux server:
`sysprog.runi.ac.il`
- Ways of accessing the server:
 - From Windows:
use remote client (such as [PuTTY](#))
 - From Mac or Linux terminal:
`ssh <username>@sysprog.runi.ac.il`
- If you wish to work locally and transfer files to the server, you can do so using a secure FTP client (such as [PuTTY SFTP](#) or [WinSCP](#))
- For detailed instructions on how to access server, see:
 - detailed log in guide on [Piazza Resources](#)
 - video guide on the course's Panoptro folder

Local Linux installation

- There are many Linux variants (flavors)
Recommended: Ubuntu, Fedora, Mint
- Each can be used in two ways
 - Full install – not simple, and somewhat risky
 - Live media – good as a demo, but has many disadvantages for real work
- It is also possible to create a **Linux virtual machine** that will operate within another OS
(using VirtualBox or VMware)
- On Windows 10, it is also possible to install a Linux shell emulator

Poll 1

How many HW exercises are in the course?

- 5
- 6
- we don't know yet

If I don't submit an exercise:

- No problem - there's the (n-1) rule.
- I will get a grade of 0 for that exercise.
- I will automatically fail the course.

If I don't find a partner for an exercise:

- I need to contact the instructors to ask permission to submit alone.
- I should ask around (on Piazza, WhatsApp, etc.) to find a partner.
- I will automatically fail the course.

Submission of the exercises is done

- Via Piazza
- Via the course server
- Via email
- On paper

Poll 1

How many HW exercises are in the course?

- 5
- **6**
- we don't know yet

If I don't submit an exercise:

- No problem - there's the (n-1) rule.
- **I will get a grade of 0 for that exercise.**
- I will automatically fail the course.

If I don't find a partner for an exercise:

- I need to contact the instructors to ask permission to submit alone.
- **I should ask around (on Piazza, WhatsApp, etc.) to find a partner.**
- I will automatically fail the course.

Submission of the exercises is done

- Via Piazza
- **Via the course server**
- Via email
- On paper

Lecture overview

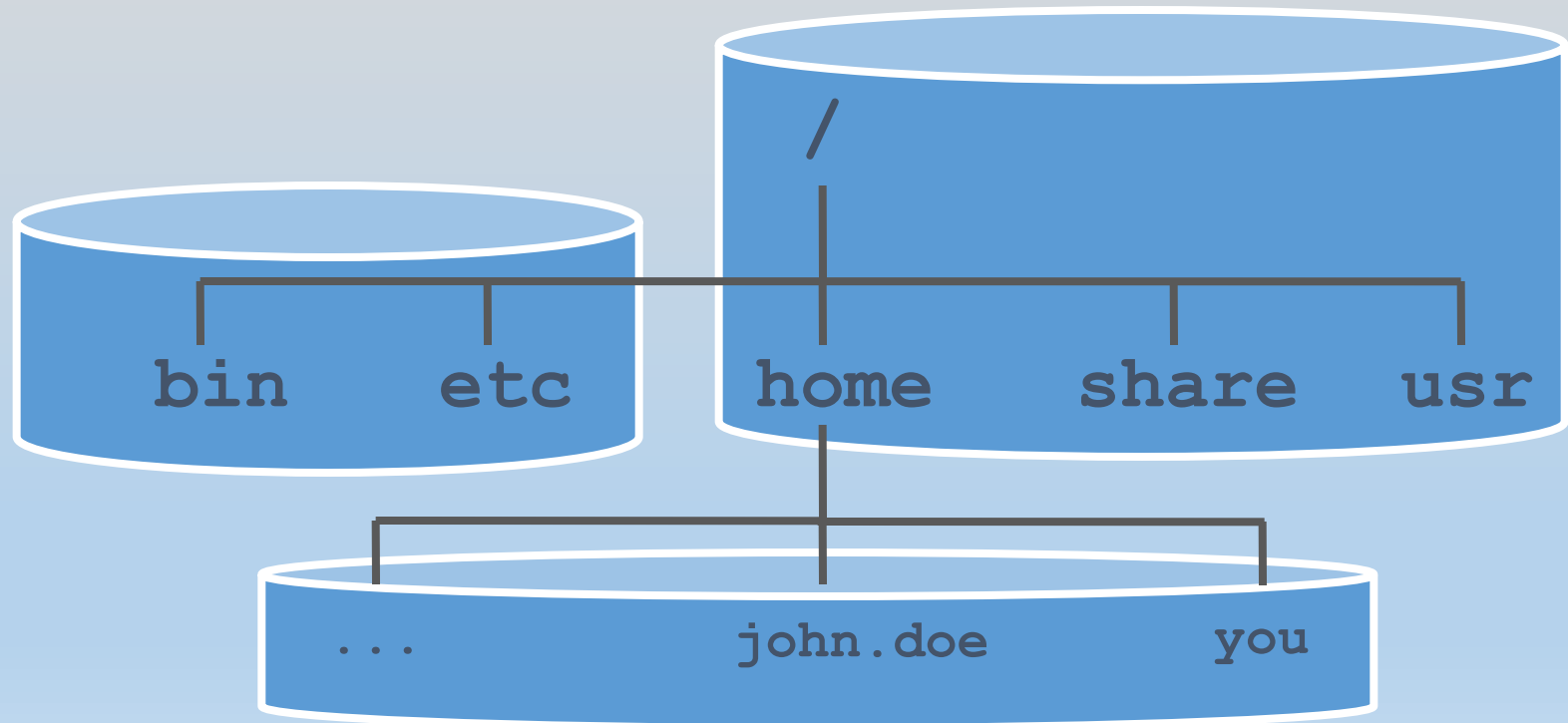
- Brief intro to course
- Course server
- **File system basics ←**
- Navigating file system (**pwd, cd, ls**)
- Manipulating files and directories
(**chmod, cp, mv, rm, mkdir, rmdir**)

Linux/UNIX file system

- Files in the Linux operating system are arranged in a hierarchical structure
- Path components are separated by a forward slash – '/' (not by a backslash)
- At the top there is the root directory: '/'
 - [Directory = folder]
- Under it are various system directories
- Each user has his/her own hierarchy of files

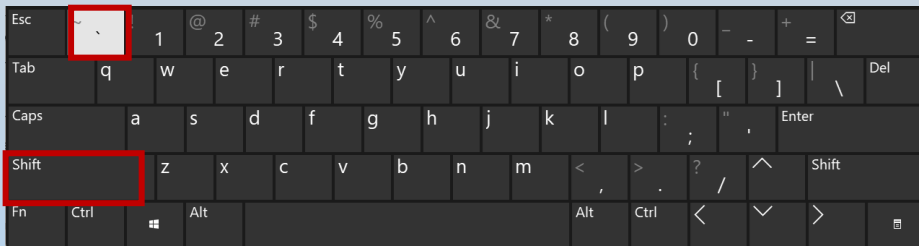
File system vs. physical disks

- The file hierarchy can be implemented using a number of physical disks

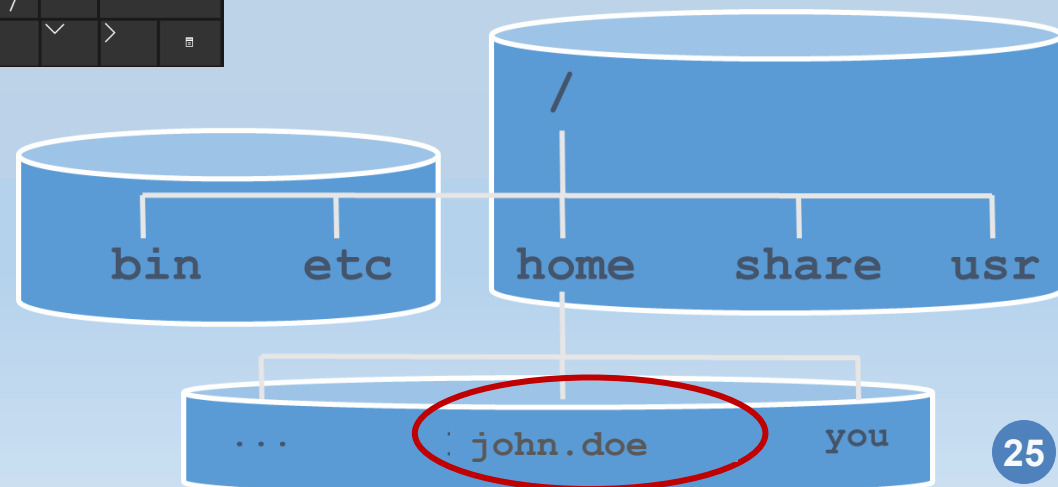


User home directory

- The home directories of users normally reside under `/home` (e.g., `/home/john.doe`)
- Assuming you are John Doe – for your own homedir, use `~`
`'~' <==> '/home/john.doe'`



← `~` created with shift + 1
(or shift + key left of 1)

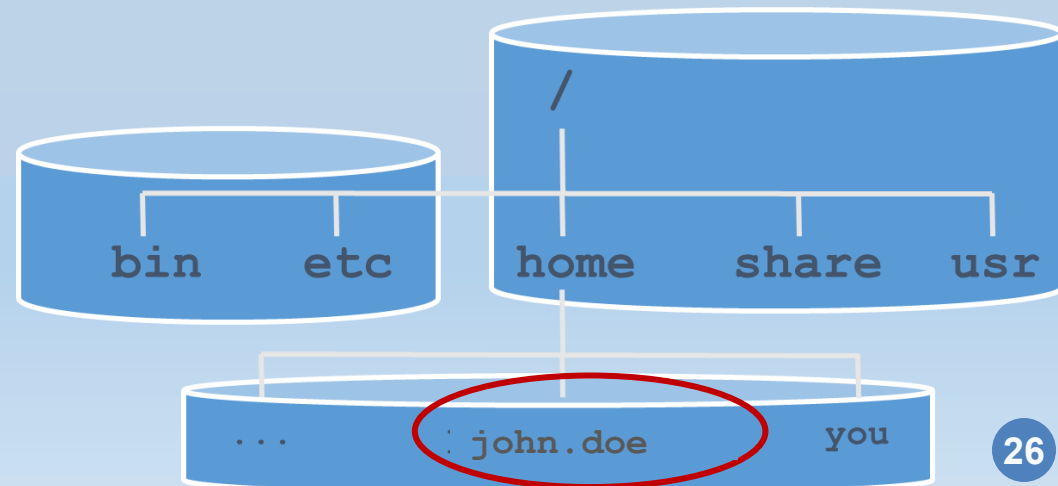


Absolute vs. relative paths

Any location in the filesystem can be specified using

- **Absolute path** from root '/'
- **Relative path** from current directory

If you are John Doe – the **absolute** path of your homedir is `/home/john.doe`



Absolute vs. relative paths

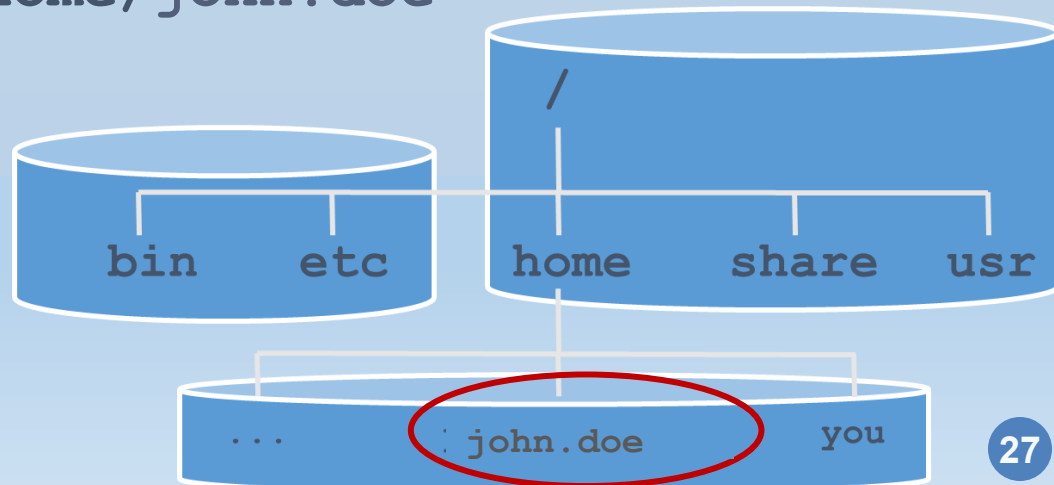
Any location in the filesystem can be specified using

- **Absolute path** from root '/'
- **Relative path** from current directory

If you are John Doe – the **relative** path of your homedir is

- Relative to /home → john.doe
- Relative to /home/john.doe → .
- Relative to /share → ../home/john.doe

. (current dir)
.. (parent dir)



Conventions for directory names

- There are **no strict directory name conventions** in Linux
- Dir names can contain almost any sequence characters, but **we will typically restrict names to letters (upper or lower case), numbers, and a few symbols (‘_’ ‘.’ ‘-’)**
 - Strict rule: don't use #, <, ?, =, +, {, \, etc.
- Dir names are **case sensitive**

Conventions for file names

- There are **no strict filename conventions** in Linux
- File names can contain almost any sequence characters, but **we will typically restrict names to letters (upper or lower case), numbers, and a few symbols (‘_’ ‘.’ ‘-’)**
 - Strict rule: don't use #, <, ?, =, +, {, \, etc.
- File names are **case sensitive**
- The file extension is the string after the last ‘.’ in the name and it typically represents the file's function (text, C code, Bash code, etc.)
- File extensions are **non-committing** and files don't have to have extensions. Changing a file's extension doesn't do anything to contents of the file.

Lecture overview

- Brief intro to course
- Course server
- File system basics
- Navigating file system (**pwd, cd, ls**) ←
- Manipulating files and directories
(**chmod, cp, mv, rm, mkdir, rmdir**)

Changing / displaying the current directory

- The **pwd** command displays the absolute path of the current directory

```
pwd
```

```
/home/john.doe/exercises
```

- The **cd** command changes the current directory. Its general form is:

```
cd [directory-name]
```

- With no parameters, **cd** changes the current directory to your home directory

ls – Listing directory contents

- The **ls** command lists the contents of some directory
- Examples:

```
ls
```

- Lists content of current directory

```
ls bin
```

- List the contents of directory **bin** (relative path)

```
ls a.txt
```

- If there exists a file called **a.txt** (relative path), then list it

```
ls a.txt bin
```

- Combination of the above

Filename expansions using wildcards

- Use symbols to define a collection of files or directories or a collection of strings
- `'*'` replaces any sequence of characters (also empty)
- `'?'` replaces a single character
- `[ ]` replaces a specific collection or range of characters
(e.g., `[rwx]` – `'r'`, `'w'`, or `'x'`,
`[r-w]` – `'r'` to `'w'`, `[a-z]` – `'a'` to `'z'`)
- `[^ ]` replaces any character except those in the specified set
(e.g., `[^rwx]` – any character other than `'r'`, `'w'`, or `'x'`)
- `{ , , }` replaces a collection of strings
(e.g., `{test,text,rest}` – matches either `'test'`, `'text'`, or `'rest'`)

Filename expansions using wildcards

Examples:

```
ls
```

```
boxes.c  DOC.txt  parse_tools.c  shape.c  tools.c  
boxes.h  HELP.txt  parse_tools.h  shape.h  tools.h
```

```
ls *.c
```

```
boxes.c  parse_tools.c  shape.c  tools.c
```

- files (and directories) whose name ends with ``.c'`

```
ls *tools*
```

```
parse_tools.c  parse_tools.h  tools.c  tools.h
```

- files (and directories) whose name contains ``tools'`

Filename expansions using wildcards

Examples:

```
ls
```

```
boxes.c  DOC.txt  parse_tools.c  shape.c  tools.c  
boxes.h  HELP.txt  parse_tools.h  shape.h  tools.h
```

```
ls *tools.?
```

```
ls *tools.[ch]
```

```
parse_tools.c parse_tools.h tools.c tools.h
```

- files (and directories) whose name has `'tools'`, which is preceded by anything but is followed by just a period and one additional character
- files (and directories) whose name has `'tools'`, which is preceded by anything but is followed by `.c` or `.h`
- (in this case, same result)

Filename expansions using wildcards

Examples:

```
ls
```

```
boxes.c  DOC.txt  parse_tools.c  shape.c  tools.c  
boxes.h  HELP.txt  parse_tools.h  shape.h  tools.h
```

```
ls {tools,shape}.*
```

```
shape.c shape.h tools.c tools.h
```

- files (and directories) whose name begins with either **'tools'** or **'shape'** and is followed by just a period and one additional character

Wildcard – important notes

- When a wildcard expression is used in the command line it is automatically converted by the Shell to the list of matching file / directory names. Thus, wildcards can be used in the context of any Linux command accepting file / directory names
- Wildcard expressions are similar to regular expressions, but they are not the same thing (more on that to come...)
- If you want to use a wildcard character (for instance ' ? ') in a string:
 - precede it with a backslash (e.g., ' \? ')
 - or wrap it with double quotes (e.g., ' "? " ')

Command line options

- Linux commands typically have optional flags they can take as input
- Flags are typically indicated by a dash (`'-'`) followed by some letter, or in long form – double dash (`'--'`) followed by a string
- The most common options for `ls` are:
 - `-a` – all (also shows **hidden files**)
 - `-F` – indicate file types
 - `-l` – long format (include file size, times, permissions, etc.)
- Options can be given separately

long form

`ls --all`

`ls --classify`

`ls --format=long`

`ls -a`

`ls -l`

or together

`ls -al`

`ls -la`

ls – Examples

```
ls
```

```
backup.tar.gz  create_backup  phone_list  temp  
bin            my_documents  solutions
```

```
ls -F
```

```
backup.tar.gz  create_backup*  phone_list  temp/  
bin/           my_documents/  solutions/
```

- File types:
 - Executable (suffix `*`)
 - Directory (suffix `/`)
 - Other (no suffix)
- File types are shown when using the `-F` option in `ls`

ls – Examples

ls

```
backup.tar.gz  create_backup  phone_list  temp
bin            my_documents  solutions
```

ls -a

```
.      .history      bin      phone_list
..     .bashrc      create_backup  solutions
.exrc  backup.tar.gz  my_documents  temp
```

- Hidden files:
 - Files and directories whose name starts with a dot (' . ') are not listed in a standard **ls** application.
 - Hidden files can be used for anything, but they are typically used for setup properties for other program:
bash (.bashrc), vi (.exrc), ssh (.ssh directory)
 - Hidden files are listed when using the **-a** option in **ls**

Poll 2

Mark all the absolute paths:

- /home/donald.trump
- exercises/ex1
- ~/exercises/ex1/my_commands.txt
- ../classes/class01/myFile.txt

Which of these are valid names for text files?

- myFile.txt
- my.file
- myFile.exe
- myFile.exe1
- myFile{3}
- dir

A directory called `dir1` contains `file1`, `file2.txt`, and `file3.c`. Which files are in the output of `ls file?.`, if the command is run from within `dir1`?

- `file1` – yes or no
- `file2.txt` – yes or no
- `file3.c` – yes or no

Which of these are valid names for directories?

- `dir`
- `my.dir`
- `file3`
- `file3.txt`

Poll 2

Mark all the absolute paths:

- `/home/donald.trump`
- `exercises/ex1`
- `~/exercises/ex1/my_commands.txt`
- `../classes/class01/myFile.txt`

Which of these are valid names for text files?

- `myFile.txt`
- `my.file`
- `myFile.exe`
- `myFile.exe1`
- `myFile{3}`
- `dir`

A directory called `dir1` contains `file1`, `file2.txt`, and `file3.c`. Which files are in the output of `ls file?.`, if the command is run from within `dir1`?

- `file1` – yes or **no**
- `file2.txt` – yes or **no**
- `file3.c` – yes or **no**

Which of these are valid names for directories?

- `dir`
- `my.dir`
- `file3`
- `file3.txt`

ls – Examples

ls

```

backup.tar.gz  create_backup  phone_list  temp
bin            my_documents  solutions
    
```

ls -l

```

-rw----- 1 john.doe students 317 Oct 20 04:35 backup.tar.gz
drwx----- 2 john.doe students 4096 Oct 19 04:31 bin
-rwxr----- 1 john.doe students 35 Oct 19 13:46 create_backup
drwx----- 2 john.doe students 4096 Oct 19 04:30 my_documents
-rw-r--r-- 1 john.doe students 17 Oct 19 04:31 phone_list
drwx----- 3 john.doe students 4096 Oct 19 04:32 solutions
-rwxrw-r-- 2 john.doe students 4096 Oct 19 04:30 temp
    
```



Permissions



Owner



Group



Size



Date/Time



Name

- The **-l** option is used for printing a detailed account for each file

File/dir permissions

The permissions have three types:

	File	Directory
Read (r)	file contents can be read and displayed	dir contents can be listed
Write (w)	file contents can be changed + file can be moved or deleted	dir attributes can be modified, e.g., can create, rename, or delete files within dir
Execute (x)	file can be executed as a program	dir can be entered into

Permissions

- Permissions are granted in three layers:
 - User/Owner – the user who created the file
 - Group – users within a specified group containing the owner
 - Others – all users in the system (other than user/owner and those in the group)



Permissions – examples

	Permissions		User	Group	Size	Date/Time	Name
1)	-rw-----	1	demo	students	317	Oct 19 04:35	backup.tar.gz
2)	drwx-----	2	demo	students	4096	Oct 19 04:31	bin
3)	-rwxr-----	1	demo	students	35	Oct 19 13:46	create_backup
4)	drwxr-----	2	demo	students	4096	Oct 19 04:30	my_documents
5)	-rw-r--r--	1	demo	students	17	Oct 19 04:31	phone_list
6)	drwxr-x---	3	demo	students	4096	Oct 19 04:32	solutions
7)	-rwxrw-r--	2	demo	students	4096	Oct 19 04:30	temp

- 1) File can be read/modified only by owner
- 2) Directory can be read/modified/entered by owner
- 3) File can be read/modified/executed by owner + read by group (those in the group “students”)
- 4) Same as 3) except directory
- 5) File can be read by anyone + modified only by owner
- 6) Directory has full access by owner + it can be read and entered by group
- 7) File can be read by anyone + modified by owner and group + executed by owner

Lecture overview

- Brief intro to course
- Course server
- File system basics
- Navigating file system (`pwd`, `cd`, `ls`)
- Manipulating files and directories
(`chmod`, `cp`, `mv`, `rm`, `mkdir`, `rmdir`) ←

chmod – Changing file permissions

- The **chmod** command changes the permissions associated with a file or directory

```
chmod [options] mode file...
```

- The format of the mode is:

```
[ugoa] [+ -] [rwx]
```

- **ugoa** user (owner), group, other, or all
 - Optional – if omitted, default is **a**
 - But – might be wonky so explicitly write **a** if that's the intention
- **+ -** add or remove permission
- **rwx** read, write, or execute

chmod – Examples

```
chmod a-w phones
```

- Remove write permissions from all users

```
chmod o+w phones
```

- Add write permissions to others

```
chmod u+x my_script addresses
```

- Make a file executable to user

```
chmod ug+r-w my_script
```

- Make a file read only to user and group

```
chmod g+w my_script
```

- Gives write permissions to group, but not to user

Note: make sure your permissions make sense

File permissions

The permission setting of a given file or directory can be represented by a binary vector of length 9

```

-rw----- 1 demo students 317 Oct 19 04:35 backup.tar.gz
110000000

drwxr-x--- 3 demo students 4096 Oct 19 04:32 solutions
111101000
    
```

- We can directly specify the permission of a given file or directory by specifying this binary vector using “octal digits” (base 8)
- Each octal digit (0-7) represents 3 binary flags for the r/w/x/ permissions of owner, group, or others.

Example:

```
chmod 644 phones
```

Owner Group Other

```

rw-r--r--
110100100
 6   4   4
    
```

File permissions – examples

		binary	octal
-rw-----	. . . backup.tar.gz	110 000 000	600
drwx-----	. . . bin	111 000 000	700
-rwxr-----	. . . create_backup	111 100 000	740
drwxr-----	. . . my_documents	111 100 000	740
-rw-r--r--	. . . phone_list	110 100 100	644
drwxr-x---	. . . solutions	111 101 000	750
-rwxrw-r--	. . . temp	111 110 100	764

chmod – common octal modes

Mode	Meaning
777	Owner, group and other can read, write, and execute file
755	Owner can read, write, and execute file; group and other can read and execute file
644	Owner can read, and write file; group and other can read file
700	Owner can read, write, and execute file; group and other have no access to the file

chmod – common command line options

- **-R**
operate recursively. Will set the permissions of all files and directories within the tree rooted at each specified directory
 - Note: -R and not -r, since -r might be confused with removing reading permission
- **-reference=[FILE]**
Copy the permissions of a given reference file to another file

chmod – additional notes

```
chmod [options] mode file...
```

Two modes of operations:

- Alphabetical (relative) permission:
- Octal (absolute) permission:

Mode = [ugoa] [+ -] [rwx]

Mode = [0-7] [0-7] [0-7]

Sensible application:

- You can assign a file with any of the $2^9=512$ possible priority modes, but there are a few rules of thumb you should follow:
 - Don't remove the owner's read access
 - Maintain hierarchical priorities – owner $\geq$ group $\geq$ others
 - If you give read access ('r') to a directory, then also give executable ('x') permissions
 - Because otherwise, may not be able to enter that directory

File manipulation commands

cp source_path dest_path

mv source_path dest_path

rm path

mkdir path

rmdir path

copy file(s)

move file(s) [copies and deletes source]

remove/delete file(s)

create new directory(ies)

delete directory(ies) [only if empty]

- All path args above can be specified as **relative** or **absolute** paths
- **source_path** and **path** can contain **one or more paths**

Examples:

==> `mkdir myDir yourDir`

creates two sub-directories `myDir` `yourDir` under current directory

==> `rm file1 file2`

deletes files `file1` `file2` from current directory

Copying directories

```
cp source_path dest_path
```

- If you wish to copy entire directories, specify the directory paths in `source_path`, and use **recursive mode** (option **-r** or **-R**)
- If `source_path` contains directory paths and recursive mode is not used, the operation will fail

Example 1:

```
==> cp hw.c classes
```

Attempts to copy `hw.c` (source) from current dir to subdir `classes` (dest). Will work, as `hw.c` is a file in the current dir and `classes` is a subdir of the current dir.

Copying directories

```
cp source_path dest_path
```

- If you wish to copy entire directories, specify the directory paths in `source_path`, and use **recursive mode** (option **-r** or **-R**)
- If `source_path` contains directory paths and recursive mode is not used, the operation will fail.

Example 2:

```
==> cp /share/ex_data .
```

Attempts to copy `ex_data` (source) from `/share` to current dir (dest).
Will fail because `ex_data` is a directory and not a file. To fix this we apply **-r**.

```
==> cp -r /share/ex_data .
```

Succeeds.

Copying directories

cp `source_path dest_path`

- If you wish to copy entire directories, specify the directory paths in `source_path`, and use **recursive mode** (option **-r** or **-R**)
- If `source_path` contains directory paths and recursive mode is not used, the operation will fail.

Example 3:

```
==> cp -r /share/ex_data file.txt ~/exercises/ex1/
```

Attempts to copy (directory) `ex_data` from `/share` (source1) and (file) `file.txt` from current dir (source2) to directory `~/exercises/ex1/`.

Will succeed if directory `~/exercises/ex1/` exists; fail otherwise.

Moving or renaming files/directories

mv source_path dest_path

- Command **mv** simply **copies and removes original**
- No need for recursive mode (**-r** or **-R**) when moving directories
- File/directory can be **renamed** by using the same directory and different file name in **source_path** and **dest_path**

Example 1:

```
==> mv ~/prog.txt ~/prog.c
```

Rename file **prog.txt** from your home directory to **prog.c**

Example 2:

```
==> mv ex1 ~
```

Move directory **ex1** from current dir to your homedir (with its contents)

Two modes for `cp` and `mv`

`cp` `source_path` `dest_path`

`mv` `source_path` `dest_path`

- **Mode I:** If `dest_path` is an **existing dir**, then all files and dirs in `source_path` are copied/moved directly into directory `dest_path` with their original names.
 - If source is a directory, then it becomes a subdirectory of dest, and if source is a file, then it becomes a file within dest.
 - Note: `source_path` may be a list of paths
- **Mode II:** If `dest_path` is **not an existing dir**, then `source_path` is copied/moved into the parent directory of `dest_path` with its name changed.
 - Note: `source_path` must be a single path, and `dest_path` must be a path **under** an existing directory.

Two modes for `cp` and `mv`

`cp` source_path dest_path

`mv` source_path dest_path

Examples:

`==> cp hw.c ~/classes/class01`

`==> cp -r ~/exercises hw.c ~/classes/class01`

Mode I

- If the dest path (`~/classes/class01`) is a path to an existing directory, then both commands are valid and will copy the source item(s) to the dest dir (`~/classes/class01`) with their original names.

Two modes for cp and mv

cp source_path dest_path

mv source_path dest_path

Examples:

```
==> cp hw.c ~/classes/class01
```

```
==> cp -r ~/exercises hw.c ~/classes/class01
```

Mode II

- If the dest path (`~/classes/class01`) does not exist:
 - The second command: must be Mode I (since Mode II is with only 1 source); **will result in an error**.
 - The first command: if `~/classes` exists, then the **first command will copy** the single source item (`hw.c`) to the parent dir (`~/classes`) with the **new name** `class01`. If even `~/classes` doesn't exist, then the **first operation will result in an error, as well**.

Two modes for `cp` and `mv` – Explanatory Example

`==> cp a.txt b/c`

- One source so can be either mode I or mode II.
- If in current dir, there is `b/c` (`c` dir within `b` dir): file `a.txt` will be copied into that directory, and it will be located at `b/c/a.txt`.
- If in current dir, there is `b` but no `b/c`: file `a.txt` will be copied into `b`, and its filename will be `b/c`.
- If in current dir, there is no `b`, there will be an **error** (since `b/c` is invalid filename – can't contain / in filename).

Delete files and directories

```
rm path  
rmdir path
```

- Command **rm** is typically used to delete files
- Command **rmdir** can delete a directory only if it is empty
- A non-empty directory can be deleted by using **rm** in recursive mode (option **-r** or **-R**)

Words of Caution

- Linux does not have an 'undo' option for filesystem commands
Once you delete a file, it is gone!
- All operations are subject to the appropriate file permissions
- Tip: remove write permissions for files you wish to protect from accidental deletion
- Use interactive mode (`-i`) to be prompted for each file that is overwritten or removed
- Use forced mode (`-f`) to remove without prompting
(for write-protected files or non-existing files)



Linux command line help

- You may get basic help on usage of a Linux command by using the option **--help**

```
ls --help
```

```
Usage: ls [OPTION]... [FILE]...
List information about the FILES (the current directory by
default).
Sort entries alphabetically if none of -cftuvSUX nor --sort is
specified.

Mandatory arguments to long options are mandatory for short
options too.
  -a, --all                do not ignore entries starting with .
  -A, --almost-all        do not list implied . and ..
  ...
```

- Square brackets ' [] ' in usage passage mean that an option or input argument is **optional** (mandatory arguments appear without brackets)

Linux command line help

- You can get detailed help on a Linux command by examining its **man** page (for manual)

man ls

```
LS (1)                                User Commands                                LS (1)

NAME
    ls - list directory contents

SYNOPSIS
    ls [OPTION]... [FILE]...

DESCRIPTION
    List
    ...
```

- The **man** pages of many Linux commands are also available on the web. See, e.g., <http://linux.die.net/man/1/>

Linux command line help

- The **man** pages of a given command can be navigated through using the following commands (from the **vi** editor):

Key	Meaning
f , <Space>	Move one page forwards
b	Move one page backwards
<Enter>	Move one line down
/pattern	Search for pattern
n	Repeat previous search
q	Quit viewing man page

Linux command line help

- A summary of common command options for lectures 1 & 2 (plus one **new** option for a command taught in lecture 2) is given in the Linux Commands guide on the [Piazza resources page](#)

Basic input / output

- The **echo** command can be used to display text

```
echo hello world
```

```
hello world
```

- Output can be redirected into a file using **>** and **>>**

```
echo hello world > file.txt  
echo hello again >> file.txt
```

file.txt will contain the following:

```
hello world  
hello again
```

- Redirection variants:
 - >** – directs output to a new file (overwrites if file exists)
 - >>** – appends output to an existing file (creates file if does not exist)

Displaying file contents

- `cat file` – displays the contents of a (text) file
- `more file` – displays the contents of a (text) file up to what can be viewed in one screen – can be scrolled down
- `less file` – similar to `more`, but you can scroll up and down and search (as you do with `man` pages)

More next week ...

Text editing in Linux

- There are several text-based text editors for Linux (`pico`, `vi`, `pine`, `emacs` ...)
- In this course, we use `pico` and `vi`
- Both are keyboard-based
- `pico` is a bit more intuitive for simple use
- `vi` is more powerful, but more difficult to get used to
- We provide starter guides for both editors on the [Piazza resources page](#)

At home...

- If you haven't received a welcome message from Piazza, email us to add you
- Review relevant guides on Piazza/watch corresponding movies on Panoptro:
 - History of Linux/C
 - Linux server (for logging in and **changing password**)
 - Linux commands (haven't yet learned all of them)
 - **pico** text editor
- Read **HW Exercises Guidelines** document
- Homework exercise #1 will be published on Thursday

